Privacy Libraries for Python, Compared

What each package does, what it assumes, and the same three jobs run in four of them

Manuel V. Loureiro

Version 2026-11-04

You hold records about people and want to publish a few statistics, train a model, or check how much a model leaks. Which Python package should do the privacy part, and what does each one quietly assume? This note surveys the packages for differential privacy, privacy auditing, encrypted computation, federated learning and anonymization, as they stood on 23 October 2026, and runs the same three jobs in four of them. The libraries agree on the arithmetic. They differ in what they make you declare, what they refuse to do, and what they let you get wrong.

Table of contents

1 What a privacy library has to get right **3**
- 1.1 The guarantee . . . 3
- 1.2 The noise a statistic needs . . . 4
- 1.3 Many releases . . . 6
- 1.4 The four duties of a library . . . 7

2 The packages that exist **7**
- 2.1 Differential privacy for statistics . . . 8
- 2.2 Differential privacy for training and accounting . . . 9
- 2.3 Synthetic data, auditing, encryption, federation and anonymity . . . 10

3 Job 1: release a count and a mean **11**
- 3.1 The arithmetic agrees . . . 13
- 3.2 Where the libraries diverge: what they make you say . . . 14
- 3.3 Where the libraries diverge: refusing . . . 15
- 3.4 The same release in the libraries not run here . . . 16

4 Job 2: train a classifier **16**
- 4.1 DP-SGD in Opacus . . . 17
- 4.2 Models in diffprivlib . . . 18
- 4.3 What the same budget buys . . . 19
- 4.4 DP-SGD in the libraries not run here . . . 20

5 Job 3: account one training run **21**

6 The rest of the toolbox **23**
6.1 Synthetic data . . . 24
6.2 Auditing . . . 24
6.3 Encrypted computation . . . 24
6.4 Federated learning . . . 25
6.5 Anonymization . . . 25

7 How to choose **25**

8 Sources **27**

Differential privacy (DP) is the standard guarantee for publishing a statistic or model computed from personal data. More than a dozen Python packages implement it, and others support related tasks, from encrypted computation to measuring what a finished model leaks. A package from the wrong family may not support the task you need. Misreading a package's assumptions can also lead you to claim a guarantee its code does not provide.

This note surveys the available packages and compares their behaviour in code. Four packages, installed at pinned versions, run the same three jobs on the same data: release a count and a mean, train a classifier, and account for one training run. OpenDP 0.16.0 and diffprivlib 0.6.6 handle statistics; Opacus 1.6.0 makes stochastic gradient descent (SGD), or DP-SGD, private; and dp-accounting 0.6.0 composes privacy costs. One script computes every number quoted in the note: code/notes/python_privacy_libraries.py.

```
uv run python code/notes/python_privacy_libraries.py
```

You need basic probability to follow the note, and nothing about privacy. Table 1 gives the question each section answers, and Table 2 lists the symbols.

Table 1: The question each section answers.

Section	The question it answers
Section 1	What must a privacy library do for you, and what can it not do?
Section 2	Which packages exist, who maintains them, and what does each need?
Section 3	How do three implementations release the same count and mean, and where do they differ?
Section 4	What does the same classifier cost at each privacy level in two training libraries?
Section 5	How far apart are the privacy accountants on one training run?
Section 6	What do the auditing, encryption, federated and anonymization packages do?
Section 7	Which package for which job, and what to check before trusting it?

Table 2: The symbols the note uses. Each is also explained where it first appears.

Symbol	Meaning
D, D'; $D \sim D'$	two neighbouring datasets: one is the other with one person's record added or removed
n	the number of records
M	a mechanism, the random procedure that computes the released output
f, Δf	a statistic, and its sensitivity: the most one record can change it
ℓ, u	the public lower and upper bounds that a library clamps values to
ε, δ	the two parameters of a differential-privacy guarantee
b	the scale of Laplace noise
σ	the noise multiplier of Gaussian noise, its standard deviation in units of the sensitivity
C	the clipping norm of DP-SGD
q, T	the sampling rate of DP-SGD, and its number of steps

1 What a privacy library has to get right

Package comparisons require a common definition of the guarantee. This section defines that guarantee and proves two results on which the packages rely. Those results imply four duties for a privacy library, which the three jobs compare.

1.1 The guarantee

Take a dataset D of n records, one per person. The running example is a public dataset of 569 breast-tumour biopsies, one record per patient, each with 30 measurements and a label that says whether the tumour was malignant. A hospital that holds such records might want to publish how many patients it saw and the average size of the tumours. Each of those numbers reveals a little about every patient, and differential privacy limits how much.

The limit compares two datasets that differ in one person. Two datasets are **neighbouring**, written $D \sim D'$, when one of them is the other with one person's record added or removed. Some libraries instead define neighbours by the replacement of one record with another. That choice changes the numbers, so a library must say which it uses, and this note uses add or remove unless it says otherwise.

A **mechanism** M is a random procedure that takes a dataset and returns an output. Its output therefore has a probability distribution, and the guarantee says that this distribution barely depends on whether any one person is present. The privacy parameter ε bounds the ratio between the probability of any set of outputs with the person and without. The slack δ is a small amount by which that bound may be exceeded.

Definition 1.1 (Differential privacy)**.** A mechanism M is (ε, δ)-differentially private if for every pair of neighbouring datasets $D \sim D'$ and every set S of outputs,

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{1}$$

When $\delta = 0$ it is ε-differentially private, also called pure DP.

Read the inequality as a bound on what an observer can learn. Whatever an observer concludes from the output, they conclude it because the output fell in some set S. Up to the slack δ, that set is at most e^{ε} times more likely with the person in the data than without. The definition covers both orders of each pair, so the bound holds both ways. So a small ε is strong privacy. At $\varepsilon = 1$ the factor is $e \approx 2.7$. At $\varepsilon = 0.1$ it is about 1.1, so the output is almost equally likely either way.

The slack δ must be far smaller than $1/n$. To see why, take a mechanism that publishes one of the n records, picked at random. The outputs that show a given person's record have probability $1/n$ with that person in the data and zero without, and no factor e^{ε} times zero can cover $1/n$. So the mechanism satisfies Definition 1.1 with $\varepsilon = 0$ and $\delta = 1/n$, and still exposes someone completely.

The guarantee is about the procedure, not about any one output, so it survives anything done afterwards. A function of a private output, computed without a second look at the data, reveals at most what the output reveals, because the observer could apply the function themselves. Libraries rely on this **post-processing** property when they round, clip or truncate what they release.

1.2 The noise a statistic needs

A mechanism hides a person by adding noise, and the amount of noise depends on the most that one person can move the statistic.

Definition 1.2 (Sensitivity)**.** The sensitivity of a statistic f, written Δf, is the most it can change between neighbouring datasets,

$$\Delta f = \max_{D \sim D'} |f(D) - f(D')|. \tag{2}$$

The maximum runs over every pair of neighbouring datasets, not only the one you hold.

The count of patients has sensitivity 1, because one patient more or less changes it by exactly one. The average tumour radius has no finite sensitivity on its own, because a dataset with one absurd value can move the average arbitrarily far. So every library makes you **clamp** the values first, which means to replace any value below a lower bound ℓ by ℓ and any value above an upper bound u by u.

The bounds must be **public**, chosen from knowledge of the measurement and not from the records. The largest value in the data is one person's value, so a bound read from it has already revealed something. After clamping, the replacement of one of n values by another moves the sum by at most $u - \ell$, so it moves the average by at most $(u - \ell)/n$. That is the sensitivity of the average when n is known. If n is not known, the average must be released as a noisy sum divided by a noisy count.

The oldest mechanism adds Laplace noise. The Laplace distribution has one parameter, its scale b, which is also the mean absolute value of a draw from it. Its density at a value z is $\frac{1}{2b}e^{-|z|/b}$, symmetric around zero, with tails that fall off exponentially. The noise must grow with the sensitivity, because it must hide a larger change, and it must grow as ε shrinks, because a stronger guarantee allows less difference between the two output distributions. The next theorem says exactly how much. Every library's Laplace mechanism implements its proof, so the proof is also the specification that Section 3 checks the libraries against.

Theorem 1.1 (Laplace mechanism)**.** *Let f be a statistic with sensitivity Δf, and let $M(D) = f(D) + Z$ with Z drawn from the Laplace distribution of scale $b = \Delta f/\varepsilon$. Then M is ε-differentially private.*

Proof. The plan is to compare the output densities under D and D' at every output, show that their ratio never exceeds e^{ε}, and then integrate over any set.

1. Adding the constant $f(D)$ shifts the Laplace density by $f(D)$. Write $p(y)$ for the density of $M(D)$ at an output y. It is the Laplace density evaluated at the distance from y to the true answer,
$$p(y) = \frac{1}{2b}\exp\left(-\frac{|y - f(D)|}{b}\right). \tag{3}$$
The density $p'(y)$ of $M(D')$ is the same expression with $f(D')$ in place of $f(D)$.
2. Both densities have the same normalizing constant, so it cancels in their ratio. What is left is a ratio of two exponentials, and its logarithm is the difference of the exponents, with the exponent of p' first because p' is the one we divide by:
$$\ln\frac{p(y)}{p'(y)} = \frac{|y - f(D')| - |y - f(D)|}{b}. \tag{4}$$
3. The top of that fraction compares how far y lies from each of the two true answers. Write $a = y - f(D')$ and $c = y - f(D)$ for the two signed distances from y to the answers. The triangle inequality in the form $|a| - |c| \le |a - c|$ says that their difference never exceeds $|f(D) - f(D')|$, the distance between the two answers themselves, which is at most Δf by Definition 1.2. So
$$\ln\frac{p(y)}{p'(y)} \le \frac{\Delta f}{b} = \varepsilon. \tag{5}$$
4. Step 3 says $p(y) \le e^{\varepsilon}p'(y)$ at every output y. The integral of both sides over any set S gives
$$\Pr[M(D) \in S] \le e^{\varepsilon}\Pr[M(D') \in S], \tag{6}$$
which is Definition 1.1 with $\delta = 0$ for this ordered pair. The argument never used which of the two datasets is larger, so it holds for the pair in the other order too.

□

The theorem fixes the error of a release before anyone sees the data. The integral of the Laplace density from t outwards on both sides gives the probability $e^{-t/b}$ that the noise exceeds t in absolute value. If that probability is 5%, then $t = b\ln 20$. That is about $3b$, so a 95% interval around the released value has half-width about three times the scale. Section 3 releases a count at $\varepsilon = 0.5$, half of a budget of 1, so its scale is 2 and its half-width is about 6 patients.

Gaussian noise is the other standard choice, for two reasons. Its cost over many releases grows more slowly. And it suits a statistic with many coordinates, such as the gradient of a model, whose change is naturally measured by its Euclidean length. For such a statistic, the sensitivity is as in Definition 1.2 but with the Euclidean length in place of the absolute value. The Gaussian mechanism adds normal noise to every coordinate, with a standard deviation of σ times the sensitivity, and the factor σ is called the **noise multiplier**.

Write Φ for the standard normal distribution function. Theorem 8 of Balle and Wang (2018) says that the mechanism is (ε, δ)-differentially private exactly when

$$\delta \geq \Phi\Big(\frac{1}{2\sigma} - \varepsilon\sigma\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{1}{2\sigma} - \varepsilon\sigma\Big). \tag{7}$$

The proof writes down the **privacy loss**, the logarithm of the ratio of the two output densities at the released output. For Gaussian noise that is a normal random variable, and the proof computes the probability that it exceeds ε. A normal variable exceeds any threshold with positive probability, so a Gaussian mechanism always carries a δ. In practice a library solves Equation 7 numerically for σ, and the accountants of Section 5 extend that computation to many steps.

1.3 Many releases

One release is rarely the end, so the next question is how the cost of several releases adds up. The total ε that a set of releases may spend is called the **privacy budget**. The basic rule is that the costs add, and the proof shows why.

Theorem 1.2 (Basic composition)**.** *Let M_1 and M_2 be two mechanisms run on the same dataset, and let the second be chosen after the first's output y_1 is known. If M_1 is ε_1-differentially private, and M_2 is ε_2-differentially private for every value of y_1, then the mechanism that releases both outputs is $(\varepsilon_1 + \varepsilon_2)$-differentially private.*

Proof.

1. A joint density of two stages is the density of the first stage times the conditional density of the second. So the density of the pair at outputs (y_1, y_2) is the density of M_1 at y_1 times the density of M_2 at y_2 given y_1.
2. Under D and D', the first factor differs by at most e^{ε_1} by the hypothesis on M_1, and the second by at most e^{ε_2} by the hypothesis on M_2 at the fixed y_1.
3. The product therefore differs by at most $e^{\varepsilon_1+\varepsilon_2}$ at every pair of outputs, and the integral over any set of pairs gives Definition 1.1 with $\varepsilon_1 + \varepsilon_2$ and $\delta = 0$.

□

This sum is correct but pessimistic. It charges every release its worst case at once, and the worst cases of independent noise draws rarely line up. An **accountant** is a procedure that computes a tighter total from the mechanisms that actually ran. For Gaussian noise, the accountants of Section 5 compute the distribution of the total privacy loss, and over k steps the ε they find grows like $\sqrt{k}$ rather than like k. A training run has hundreds of steps, so the plain sum is too loose to use, and the accountant that a library ships decides what ε you can report.

Those accountants serve **DP-SGD**, the private form of SGD. To train a model is to adjust its parameters step by step to reduce its error on the training records. Each step moves the parameters along the **gradient**, the direction in which that error falls fastest. DP-SGD runs a fixed number T of such steps.

At each step, DP-SGD draws a random batch in which every record appears independently with probability q, the **sampling rate**, which is called Poisson sampling. It computes one gradient per record in the batch, shrinks each to a Euclidean length of at most C, the **clipping norm**, sums them, and adds Gaussian noise with standard deviation σC to every coordinate. Clipping fixes the sensitivity of the sum at C, since one record more or less changes it by one clipped gradient. Sampling makes each step cost less than Equation 7 says, because a record takes part only with probability q. The accountant turns σ, q and T into one ε for the whole run.

1.4 The four duties of a library

The results above, together with what is known to go wrong when people implement them, give a library four duties. Each duty is a place where hand-written code fails, and Section 7 turns them into checks.

1. **Bound the sensitivity.** Clamp values into public bounds, know or bound n, and, when one person can contribute several records, bound how many. The last matters because the neighbouring relation must then cover all of that person's records at once.
2. **Sample the noise correctly.** A computer's floating-point numbers are a finite set with gaps between them, so a floating-point Laplace sample is not a real-valued one. Mironov (2012) showed that the pattern of outputs that can occur depends on the true answer, so an attacker can tell $f(D)$ from $f(D')$ exactly in a mechanism that is ε-private on paper. OpenDP implements its noise against that attack, and diffprivlib's documentation says its Laplace mechanism is safe too. This note does not test either claim, and a hand-written `rng.laplace(scale=b)` makes no such claim at all.
3. **Account.** Add up what the releases or steps have spent, with the tightest theorem that applies to what the code actually did. The libraries differ most here, and Section 5 measures how much.
4. **Refuse.** A privacy budget is a limit, and a library that keeps answering after the limit is only a noise generator.

2 The packages that exist

Which package fits a job depends first on its family, so this section lists the packages by job. Every version, date and licence comes from the Python Package Index, the public catalogue that `pip install` draws from, and from each project's repository, read on 23 October 2026. A date is that of the latest release. A package with rare releases can still be alive, so the text under each table says how active its repository is.

2.1 Differential privacy for statistics

Table 3 lists the packages that release counts, sums, means, quantiles and histograms. The text after it adds what the table cannot show, which is how each package treats a person with many records.

Table 3: Packages that release statistics.

Package	Maintainer	Latest release	Licence	Needs	What it gives you
`opendp`	The OpenDP project	0.16.0, October 2026	MIT	Python 3.10 or later; Polars optional	Transformations and measurements with written proofs in a Rust core; a `Context` that holds the budget; DP queries on Polars dataframes
`diffprivlib`	IBM	0.6.6, April 2025	MIT	NumPy, scikit-learn 1.4 or later	NumPy-style tools, a dozen mechanisms, scikit-learn-style models and a `BudgetAccountant`
`pipeline-dp`	OpenMined and Google	0.3.1, June 2026	Apache-2.0	Python 3.11 to 3.13; Apache Beam or Spark optional	Pipeline aggregations with contribution bounding and partition selection built in
`tmlt.analytics`	OpenDP, originally Tumult Labs	0.21.0, June 2026	Apache-2.0	PySpark and Java; Python 3.10 to 3.12	A `QueryBuilder` over Spark dataframes, with private joins and stability tracking in `tmlt.core`
`python-dp`	OpenMined	1.1.5, May 2026	Apache-2.0	Linux x86-64 or macOS wheels; Python 3.11 or later	Bindings to Google's C++ building blocks: bounded mean, sum and quantiles with Laplace noise
`smartnoise-sql`	OpenDP	1.0.10, April 2026	MIT	OpenDP 0.14, pandas or a SQL connection	DP SQL queries over a schema file, with a per-session budget

OpenDP has a formal programming framework behind it. Every operation is either a **transformation**, a step from data to data such as clamping, or a **measurement**, a step that releases a noisy output. Each transformation comes with a proof of how much one record can change its output, and each measurement with a proof of its privacy cost. A chain of steps composes their proofs, which is how OpenDP's `Context` knows the cost of a query it has never seen. For a person with many records, you name an identifier column, and the proofs then count all of that person's

records as one change. Its repository is active, and its README commits to named maintainers and response times.

Tumult Analytics has the same framework idea on Spark, with the same identifier mechanism. Its history matters if you choose it. Tumult Labs' engineering team joined LinkedIn in 2025, and in October 2025 the OpenDP project took over the maintenance of Tumult's open-source libraries, whose repositories remain active.

diffprivlib, the library that looks most like NumPy, assumes one record per person and leaves any bound on contributions to the caller. Its last release was in April 2025. Its repository merged an import fix for scikit-learn 1.9 in October 2026, but the model failure that Section 4 reports is still on its main branch, and its README says the public release is for research and education.

PipelineDP and PyDP take contribution bounds as arguments of every aggregation. PipelineDP's README calls the project experimental and advises against production use, although Google's own repository lists it as suitable for production, and this note takes the maintainers' word. PyDP has one release in 2026 and depends on wheels that exist only for Linux x86-64 and macOS. SmartNoise SQL pins OpenDP below 0.15, so it cannot be installed next to the OpenDP 0.16.0 that this note runs.

2.2 Differential privacy for training and accounting

Table 4 lists the packages that train models with DP-SGD, and the one that only does the accounting. Section 4 and Section 5 explain the techniques that its cells name.

Table 4: Packages that train models and account for training.

Package	Maintainer	Latest release	Licence	Needs	What it gives you
`opacus`	Meta	1.6.0, May 2026	Apache-2.0	PyTorch 2.6 or later	A `PrivacyEngine` that wraps model, optimizer and loader: Poisson sampling, per-example clipping, Gaussian noise, three accountants; memory-saving clipping for large models
`tensorflow-privacy`	Google	0.9.0, February 2024	Apache-2.0	TensorFlow 2.4 to 2.15, Python 3.9 to 3.11	DP optimizers and Keras models, adaptive clipping, tree aggregation, Rényi accounting
`jax-privacy`	Google DeepMind	2.0.0, April 2026	Apache-2.0	JAX 0.7.1 or later	DP-SGD building blocks with independent or correlated noise, batch selection, accounting through `dp-accounting`, auditing with canaries

Package	Maintainer	Latest release	Licence	Needs	What it gives you
`dp-accounting`	Google	0.6.0, February 2026	Apache-2.0	NumPy and SciPy only	`DpEvent` descriptions of mechanisms and compositions; Rényi and privacy-loss-distribution accountants that turn them into (ε, δ)

Opacus is the library of choice for PyTorch, and it keeps moving. TensorFlow Privacy's repository is active too, but its last release, in February 2024, requires TensorFlow 2.15 or older and Python 3.11 or older, so it does not install on a current stack. That release also moved its membership-inference module into a separate package, `tensorflow-empirical-privacy`. JAX Privacy was rewritten for its 2.0 release, and its documentation recommends that you install it from the repository head, because the head has features that the release lacks. dp-accounting is shared infrastructure. TensorFlow Privacy and JAX Privacy delegate their accounting to it, while Opacus has its own, so dp-accounting is an independent check on Opacus in Section 5.

2.3 Synthetic data, auditing, encryption, federation and anonymity

Table 5 lists the remaining families. None of them was run for this note, so Section 6 describes them after the three jobs.

Table 5: Packages for synthetic data, auditing, encrypted and federated computation, and anonymization.

Package	Family	Maintainer	Latest release	Licence	Needs
`smartnoise-synth`	DP synthetic data	OpenDP	1.0.8, April 2026	MIT	Opacus below 0.15, `mbi`, PyTorch
`synthcity`	Synthetic data, some DP	van der Schaar lab	0.2.12, May 2025	Apache-2.0	PyTorch 2.1 or 2.2, NumPy 1
`tensorflow-empirical-privacy`	Membership-inference tests	Google	0.1.0, February 2024	Apache-2.0	TensorFlow 2.4 to 2.15, Python 3.9 to 3.11
`privacy-meter`	Membership-inference audits	National University of Singapore	1.0.1, July 2023; the repository is newer	MIT	PyTorch
`adversarial-robustness-toolbox`	Attacks, including inference	IBM, now under the Linux Foundation	1.20.1, July 2025	MIT	Any of the major frameworks

Package	Family	Maintainer	Latest release	Licence	Needs
`tenseal`	Homomorphic encryption	OpenMined	0.3.18, September 2026	Apache-2.0	Python 3.11 or later
`crypten`	Secure multi-party computation	Meta	0.4.1, December 2022; repository archived	MIT	PyTorch, Linux or macOS
`concrete-ml`	Fully homomorphic encryption for ML	Zama	1.9.0, April 2025	BSD-3-Clause-Clear, plus a patent licence for commercial use	Python 3.8 to 3.12
`syft`	Remote data science	OpenMined	0.10.0, August 2026	Apache-2.0	Python 3.10 or later; Google Drive as transport
`flwr`	Federated learning	Flower Labs	1.42.0, October 2026	Apache-2.0	Python 3.11 or later; any framework
`tensorflow-federated`	Federated learning	Google	0.87.0, September 2024	Apache-2.0	Python 3.9 to 3.11 on the index
`nvflare`	Federated learning	NVIDIA	2.9.0, September 2026	Apache-2.0	Python 3.10 or later
`anjana`	k-anonymity and relatives	Instituto de Física de Cantabria	1.2.3, August 2026	Apache-2.0	pandas
`pycanon`	Anonymity measurement	Instituto de Física de Cantabria	1.3.6, August 2026	Apache-2.0	pandas

3 Job 1: release a count and a mean

The first job is the release of two statistics, and it shows what each implementation makes you declare. Release how many patients the dataset holds and the average of the `mean radius` column, with a total budget of $\varepsilon = 1$ and no δ, half to each query. The bounds of the column, 6.981 and 28.11, come from the dataset's published description, which lists the minimum and maximum of every feature. They stand in for the domain knowledge a hospital would have, such as the range

of tumour sizes its instruments can measure. The stand-in is not perfect, because the published summary was computed from these very records, and a real release must take its bounds from knowledge that is independent of them.

The scales follow from Theorem 1.1. The count has sensitivity 1, so at $\varepsilon = 0.5$ its scale is 2. The mean has sensitivity 21.129/569, the width of the bounds over the number of records, so its scale is $21.129/(569 \times 0.5) \approx 0.0743$. Use the textbook formula implemented with NumPy as the baseline for comparing the libraries:

```
def by_hand(values: Array, bounds: tuple[float, float], eps: float, rng:
↪  np.random.Generator) -> Release:
    """Textbook Laplace mechanism in NumPy: half the budget to the count, half to the
    ↪  clamped mean."""
    n = len(values)
    count_scale, mean_scale = laplace_scales(n, bounds, eps / 2)
    count = n + rng.laplace(scale=count_scale)
    mean = float(np.clip(values, *bounds).mean()) + rng.laplace(scale=mean_scale)
    return Release(count, mean, count_scale, mean_scale)
```

OpenDP works differently. You declare the data, the unit of privacy and the total budget once, in a `Context`, and say how many queries will share the budget, which is the `split_evenly_over` argument. Each query is then a chain of transformations that ends in a noise mechanism whose scale you leave blank, and OpenDP finds the scale that spends exactly the query's share:

```
def with_opendp(values: Array, bounds: tuple[float, float], eps: float) -> Release:
    """OpenDP's Context API: the budget is declared once, and each query takes an even
    ↪  share of it."""
    context = dp.Context.compositor(
        data=[float(v) for v in values],
        privacy_unit=dp.unit_of(contributions=1),  # neighbours add or remove one
↪  person's record
        privacy_loss=dp.loss_of(epsilon=eps),
        split_evenly_over=2,
        domain=dp.vector_domain(dp.atom_domain(T=float, nan=False)),
    )
    count = vector_query(context).count().laplace()  # OpenDP picks the scale that
↪  spends exactly eps/2
    mean = vector_query(context).clamp(bounds).resize(size=len(values),
↪  constant=bounds[0]).mean().laplace()
    return Release(count.release(), mean.release(), count.param(), mean.param())
```

The `domain` declares that the data are floats with no missing values, and OpenDP refuses to build the mean without it. A missing value, or `NaN`, would pass through clamping unchanged and invalidate the proof. The `resize` step declares the number of records n. It pads the data with a constant or truncates it to that size, allowing the proof to treat the size as fixed and the mean's sensitivity to be $(u - \ell)/n$. Here the size is public because the dataset contains 569 published records. Without `resize`, OpenDP would reject a Laplace mean because a mean over an unknown number of records has no finite sensitivity.

diffprivlib has a NumPy-style interface. Each function, which its documentation calls a tool, takes its own ε. An optional `BudgetAccountant` composes the costs and rejects calls that would exceed

the budget. For these finite data, the count call selects every record with the condition `values > -np.inf`:

```
def with_diffprivlib(values: Array, bounds: tuple[float, float], eps: float, seed: int)
 ↪  -> Release:
    """diffprivlib's NumPy-like tools: each call takes its own epsilon, and a
     ↪  BudgetAccountant adds them up."""
    accountant = BudgetAccountant(epsilon=eps)  # refuses a query once the total would
 ↪  pass eps
    rng = np.random.RandomState(seed)
    count = dp_tools.count_nonzero(values > -np.inf, epsilon=eps / 2, random_state=rng,
 ↪  accountant=accountant)
    mean = dp_tools.mean(values, epsilon=eps / 2, bounds=bounds, random_state=rng,
 ↪  accountant=accountant)
    count_scale, mean_scale = laplace_scales(len(values), bounds, eps / 2)  # what its
 ↪  mechanisms use
    return Release(float(count), float(mean), count_scale, mean_scale)
```

Nothing tells diffprivlib what n is. Its `mean` reads the length of the array and uses the sensitivity $(u - \ell)/n$, which is correct only if n is public. Its documentation does not state that assumption, although the library does warn when the bounds are missing.

3.1 The arithmetic agrees

Table 6: The count and the mean of the mean radius column at $\varepsilon = 1$, half to each query, in three implementations. The half-width is for Laplace noise of the mean's scale.

Implementation	Count scale	Mean scale	95% half-width of the mean	What it had to be told
By hand (NumPy)	2.0000	0.0743	0.222	the bounds and n, and the formula for each scale
OpenDP	2.0000	0.0743	0.222	the bounds, n for the mean, the unit (one record) and the total budget
diffprivlib	2.0000	0.0743	0.222	the bounds and an ε per call; n it reads from the array

Table 6 shows that all three implementations arrive at the same scales, 2 for the count and 0.0743 for the mean. OpenDP's scale for the mean is larger than the textbook value by a few parts in a trillion, which the table rounds away. Its floating-point mechanism rounds the scale up to cover the rounding of the arithmetic, as the duty to sample the noise correctly requires. OpenDP also reports the accuracy of a release before it makes it, and its 95% half-width for the count is 5.99, which is $2 \ln 20$.

The script next compares the noise across 2,000 releases from each implementation. It measures error as the released value minus the true value, then prints each query's mean absolute error and the 95th percentile of absolute error. For Laplace noise of scale b, the textbook values are b

and $b \ln 20$. OpenDP's noise cannot be seeded, so the output below shows one run; its row varies between runs within sampling error:

```
== empirical errors over 2000 releases (mean |error|, 95th percentile), textbook b and
 ↪  b ln 20
textbook: count 2.000 5.991  mean 0.0743 0.2225
By hand (NumPy)  count 2.035 6.082  mean 0.0746 0.2337
OpenDP           count 1.881 6.000  mean 0.0743 0.2225
diffprivlib      count 0.969 5.000  mean 0.0736 0.2252
```

The mean errors are similar across implementations, but the count errors differ because the libraries use different noise. OpenDP adds integer noise from the discrete Laplace distribution, whose mass at each integer k is proportional to $e^{-|k|/b}$. Its quantiles are therefore whole numbers. At scale 2, its mean absolute noise is $2e^{-1/2}/(1-e^{-1}) \approx 1.92$, rather than 2; the observed value of 1.88 is within sampling error.

diffprivlib's count error is half of everyone else's, because its counting mechanism clips its output to the range from 0 to n. A count of every record sits exactly at n, so every positive noise draw is clipped back to the true value. Clipping the output is post-processing, so the guarantee holds. But the released count can never exceed the truth, and an analyst who assumes symmetric noise will misread it. The noise a library adds is a design decision that you have to read about, not a formula that you can assume.

3.2 Where the libraries diverge: what they make you say

The implementations differ in which assumptions they require you to declare. The hand-written version needs only the formula. If its author forgot to clamp the data, it would use the wrong sensitivity without warning. diffprivlib requires bounds but silently assumes that n is public. OpenDP requires the bounds, n, the domain and the privacy unit, and refuses to continue when any is missing.

OpenDP checks that your declarations are consistent, but it cannot verify that they are true. Its Polars interface runs DP queries on dataframes and lets you declare public facts about the data, such as the number of records. OpenDP calls such a fact an `invariant` of a `Margin`, its term for known information about a group of rows. Here the record count is already published, so the script declares it public and limits each person to one contribution:

```
def opendp_public_length(values: Array, bounds: tuple[float, float], eps: float,
 ↪  contributions: int) -> float:
    """The noise scale OpenDP's Polars plan gives the sum inside a mean when the number
     ↪  of records is declared
    public (`invariant="lengths"`), for a privacy unit of `contributions` rows."""
    frame = pl.DataFrame({COLUMN: values}).lazy()
    context = dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=contributions),
        privacy_loss=dp.loss_of(epsilon=eps),
        split_evenly_over=1,
        margins=[dp.polars.Margin(by=[], max_length=len(values), invariant="lengths")],
```

```
    )
    column: Any = pl.col(COLUMN)  # typed Any because OpenDP adds the `.dp` namespace
↪   to Polars expressions at run time
    query = context.query().select(column.dp.mean(bounds))
    plan = query.summarize()  # one row per noisy quantity: the sum and the length
    return float(plan.filter(pl.col("aggregate") == "Sum")["scale"].item())
```

```
== OpenDP with the number of records declared public: scale of the sum inside the mean
contributions=1: sum scale 4.042e-09  (as a mean: 7.103e-12; textbook (high-low)/(n
↪   eps) = 0.03713)
contributions=2: sum scale 21.13  (as a mean: 0.03713; textbook (high-low)/(n eps) =
↪   0.03713)
```

The script prints the noise scale of the sum inside the mean, that scale divided by n, and the textbook scale $(u-\ell)/(n\varepsilon)$ of the mean at the full $\varepsilon = 1$ of this one query. With one contribution and a public length, OpenDP adds noise of scale 4×10^{-9} to a sum in the thousands, which is no noise at all. Yet the guarantee it states is true, and that is the trap.

The guarantee is true because it is empty. If the number of records is fixed, no dataset of the same size can differ from this one by the addition or removal of a single record, and any claim about an empty set of neighbours holds. A public length matches neighbours that differ by the replacement of one record. OpenDP counts the distance between two datasets as the records added plus the records removed, so a replacement is two contributions. With two contributions, the sum's scale at $\varepsilon = 1$ is 21.13, the width of the bounds, because the replacement of one clamped value moves the sum by at most $u - \ell$. A framework that proves everything from your declarations is exactly as safe as your declarations, and OpenDP's documentation warns you to declare them conservatively.

3.3 Where the libraries diverge: refusing

The fourth duty is to stop when the budget is spent. The script spends the whole budget on two queries and then asks each library for a third:

```
== a third query after the budget is spent
OpenDP: ValueError: Privacy allowance has been exhausted
diffprivlib: BudgetError: Privacy spend of (0.5,0) not permissible; will exceed
↪   remaining privacy budget.
```

Both refuse, but they keep the budget in different places. In OpenDP the budget belongs to the `Context`, which holds the data, so you cannot query the data outside it. In diffprivlib it belongs to an accountant object that you must pass to every call, and a call without it spends from a default accountant with an infinite budget. The hand-written version never refuses.

3.4 The same release in the libraries not run here

Tumult Analytics and PipelineDP were not run, because the first needs Spark and Java and the second a pipeline runtime or its own local backend. Their code for this job still shows their design. Tumult Analytics declares the unit of privacy, which it calls a protected change (here `AddOneRow`), on a session that holds the dataframe, and each query spends its own share of the budget. The code below follows its documentation for version 0.21.0 and was not run:

```
from tmlt.analytics import AddOneRow, PureDPBudget, QueryBuilder, Session

session = Session.from_dataframe(
    privacy_budget=PureDPBudget(epsilon=1), source_id="patients", dataframe=frame,
 ↪  protected_change=AddOneRow()
)
count = session.evaluate(QueryBuilder("patients").count(), PureDPBudget(epsilon=0.5))
mean = session.evaluate(
    QueryBuilder("patients").average("radius", low=6.981, high=28.11),
 ↪  PureDPBudget(epsilon=0.5)
)
```

PipelineDP asks for the same facts in other words. It wants to know how to extract a person's identifier from a row, which group, or partition, the row belongs to, and the value to aggregate. It then enforces the contribution bounds you give it, here one row per person in one partition. The code below follows its documentation for version 0.3.1 and was not run:

```
import pipeline_dp

budget = pipeline_dp.NaiveBudgetAccountant(total_epsilon=1, total_delta=1e-6)
engine = pipeline_dp.DPEngine(budget, pipeline_dp.LocalBackend())
params = pipeline_dp.AggregateParams(
    metrics=[pipeline_dp.Metrics.COUNT, pipeline_dp.Metrics.MEAN],
    max_partitions_contributed=1, max_contributions_per_partition=1, min_value=6.981,
 ↪  max_value=28.11,
)
extractors = pipeline_dp.DataExtractors(
    privacy_id_extractor=lambda row: row.patient, partition_extractor=lambda row:
 ↪  "all",
    value_extractor=lambda row: row.radius,
)
result = engine.aggregate(rows, params, extractors, public_partitions=["all"])
budget.compute_budgets()
```

Both make the unit of privacy a required argument, the protected change in one and the identifier extractor in the other. OpenDP shares that design choice, and diffprivlib leaves it to the caller.

4 Job 2: train a classifier

The second job asks what the same privacy level costs a classifier in two training libraries. The task is to predict whether a tumour is malignant from its 30 measurements. The data are split

once, with a fixed seed, into 455 training and 114 test records. Every feature is scaled into the unit interval with its published minimum and maximum, with the caveat of Section 3. Fourteen raw values fall outside their published bounds, because the published summary is rounded, so clamping moves them onto the bounds.

The value of δ follows a rule. With n records, this note takes δ at most $1/n^{1.1}$, rounded down to a power of ten, which is the convention of Ponomareva et al. (2023). For 455 records that gives 10^{-3}. It is below $1/n$, which is 2.2×10^{-3}, but not far below, which is the price of a small dataset.

Three models are compared, a logistic regression trained with DP-SGD, a naive Bayes classifier and a random forest. Each is trained at five privacy levels, from $\varepsilon = 0.1$ to $\varepsilon = 10$, and twenty times at each level with different random seeds, because DP training is random and one run can be lucky.

4.1 DP-SGD in Opacus

Opacus turns an ordinary PyTorch training loop into DP-SGD with one call. The model here is a logistic regression, the simplest classifier, which scores a record by a weighted sum of its features. Training runs for 20 epochs, or passes over the training records, in batches of 32 records on average. The clipping norm is 1, and the learning rate, the size of each step along the gradient, is 0.5:

```
def train_dp_sgd(data: Data, sigma: float | None, seed: int, delta: float) ->
 ↪  tuple[float, float | None]:
    """A logistic regression trained by SGD in PyTorch; with a noise multiplier, Opacus
     ↪  makes it DP-SGD."""
    torch.manual_seed(seed)
    x = torch.tensor(data.x_train, dtype=torch.float32)
    y = torch.tensor(data.y_train)
    loader = torch.utils.data.DataLoader(torch.utils.data.TensorDataset(x, y),
 ↪  batch_size=BATCH, shuffle=True)
    model: nn.Module = nn.Linear(x.shape[1], 2)
    optimizer: torch.optim.Optimizer = torch.optim.SGD(model.parameters(), lr=LR,
 ↪  momentum=MOMENTUM)
    engine: PrivacyEngine | None = None
    if sigma is not None:
        engine = PrivacyEngine(accountant="prv")
        model, optimizer, loader = engine.make_private(  # Poisson sampling,
 ↪  per-example clipping, Gaussian noise
            module=model, optimizer=optimizer, data_loader=loader,
 ↪  noise_multiplier=sigma, max_grad_norm=CLIP
        )
    for _ in range(EPOCHS):
        for xb, yb in loader:
            optimizer.zero_grad()
            nn.functional.cross_entropy(model(xb), yb).backward()
            optimizer.step()
    with torch.no_grad():
        predicted = model(torch.tensor(data.x_test,
 ↪  dtype=torch.float32)).argmax(dim=1).numpy()
    return float((predicted == data.y_test).mean()), engine.get_epsilon(delta) if
     ↪  engine else None
```

The call `make_private` returns replacements for the model, the optimizer and the loader. The wrapped model keeps one gradient per record, and the optimizer clips those gradients and adds the noise. The loader draws Poisson batches, whose size varies from step to step, because that is the sampling the accountant assumes.

The noise multiplier σ is not chosen by hand. A helper of Opacus, `get_noise_multiplier`, searches for the σ that makes the run meet the target ε at $\delta = 10^{-3}$. Opacus sets the sampling rate q to one over the number of batches per epoch, $1/\lceil 455/32 \rceil = 1/15$, so an epoch is 15 steps and the run is 300 steps. For $\varepsilon = 1$ the search gives a noise multiplier of 3.145. More privacy needs more noise, so the multiplier rises to 22.5 at $\varepsilon = 0.1$ and falls below 1 at $\varepsilon = 10$.

4.2 Models in diffprivlib

diffprivlib offers scikit-learn-style models whose `fit` is private, from naive Bayes and logistic regression to k-means and a random forest. Its logistic regression would be the natural comparison with DP-SGD, because it trains the same model. It uses objective perturbation, which adds a random linear term to the loss instead of noise to each gradient. But it cannot run for this note. Its constructor passes an argument named `multi_class` to scikit-learn. Version 1.5 of scikit-learn deprecated that argument and announced its removal in 1.8. With the scikit-learn 1.8.0 pinned for this note, the model fails before it sees any data:

```
diffprivlib LogisticRegression with scikit-learn 1.8.0: TypeError:
 ↪  LogisticRegression.__init__() got an
unexpected keyword argument 'multi_class'
```

The repository's main branch, read the day this note was written, still passes the argument, so the unreleased version fails too. This is what a slow release cadence costs a library that wraps a fast-moving one.

The two diffprivlib models that do run are naive Bayes and the random forest. A naive Bayes classifier describes each class by the mean and variance of every feature within it, and diffprivlib's version adds Laplace noise to each of those numbers. A random forest votes among many decision trees, each a sequence of threshold tests on single features. diffprivlib's version builds the trees with random tests that never look at the data, and lets each leaf vote with noisy class counts:

```
def train_naive_bayes(data: Data, eps: float | None, seed: int) -> float:
    """scikit-learn's Gaussian naive Bayes, or diffprivlib's, which perturbs each
     ↪  class's means and variances."""
    d = data.x_train.shape[1]
    if eps is None:
        return GaussianNB().fit(data.x_train, data.y_train).score(data.x_test,
         ↪  data.y_test)
    model = DPGaussianNB(epsilon=eps, bounds=(np.zeros(d), np.ones(d)),
 ↪  random_state=seed)  # public bounds
    return model.fit(data.x_train, data.y_train).score(data.x_test, data.y_test)
```

```
def train_random_forest(data: Data, eps: float | None, seed: int) -> float:
    """scikit-learn's random forest, or diffprivlib's, whose trees are built blind and
     ↪  whose leaves vote with noise."""
    d = data.x_train.shape[1]
    if eps is None:
        return RandomForestClassifier(random_state=seed).fit(data.x_train,
         ↪  data.y_train).score(data.x_test, data.y_test)
    model = DPRandomForest(epsilon=eps, bounds=(np.zeros(d), np.ones(d)), classes=[0,
 ↪  1], random_state=seed)
    return model.fit(data.x_train, data.y_train).score(data.x_test, data.y_test)
```

Both models give pure ε-DP. A guarantee with $\delta = 0$ is also a guarantee with $\delta = 10^{-3}$, since Definition 1.1 only gets weaker as δ grows, so the three models share one axis.

4.3 What the same budget buys

Figure 1 shows the test accuracy of each model against ε, averaged over the twenty seeds, with a band from the 10th to the 90th percentile, and each model's accuracy without privacy as a dashed line. Table 7 gives the averages.

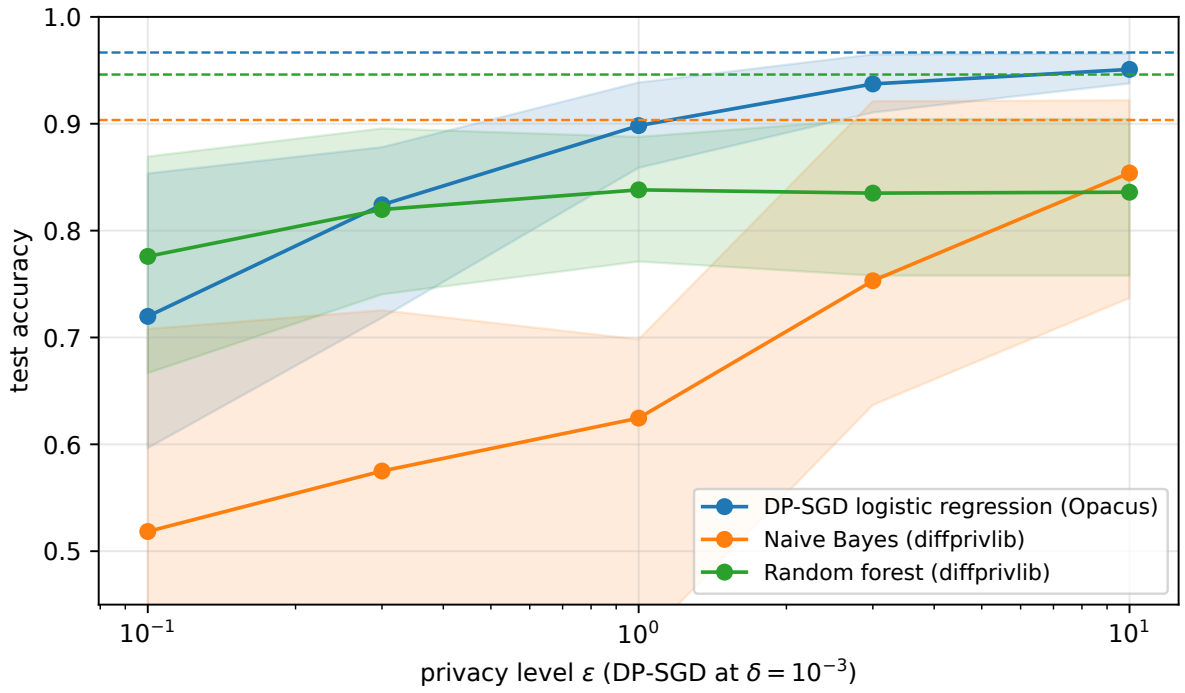


Figure 1: Test accuracy against the privacy level for three private models, each averaged over twenty seeds with its 10th to 90th percentile band. Each dashed line is the same model trained without differential privacy.

Table 7: Mean test accuracy over the seeds, by technique and privacy level; DP-SGD at $\delta = 10^{-3}$, the diffprivlib models with pure ε-DP.

Technique	$\varepsilon = 0.1$	$\varepsilon = 0.3$	$\varepsilon = 1$	$\varepsilon = 3$	$\varepsilon = 10$	No DP
DP-SGD logistic regression (Opacus)	0.720	0.824	0.898	0.937	0.951	0.967
Naive Bayes (diffprivlib)	0.518	0.575	0.625	0.753	0.854	0.904
Random forest (diffprivlib)	0.776	0.820	0.838	0.835	0.836	0.946

The DP-SGD curve climbs from 0.720 at $\varepsilon = 0.1$ to 0.951 at $\varepsilon = 10$, compared with 0.967 without privacy. At $\varepsilon = 1$, it reaches 0.898, seven percentage points lower; this is the observed accuracy cost on the 455-record dataset. The noise is added to the sum of a batch's gradients and then divided by the batch size, so a larger batch dilutes it. However, a larger batch increases the sampling rate and therefore costs more privacy per step, while a small dataset leaves little room to increase batch size. For every run calibrated to $\varepsilon = 1$, Opacus's accountant reports $\varepsilon = 0.995$ because the search for σ stops within a tolerance of the target. Publish that reported value.

The naive Bayes model does badly at every privacy level. A classifier that always predicts the majority class scores 0.632 on this test set, and the private naive Bayes stays below that until $\varepsilon = 3$. The reason is how it spends the budget. diffprivlib gives one third of ε to the class counts, one third to the feature means and one third to the feature variances, and divides each of the last two among the 30 features, so each mean and each variance gets $\varepsilon/90$. The two classes do not split it further, because each record belongs to one class, so the same budget covers both. That leaves sixty independent noise draws per class, many more things to protect than one model, which also makes its band the widest.

The random forest stops at a plateau. From $\varepsilon = 1$ upwards it sits near 0.84, eleven points below the same forest without privacy, and more budget does not help. Its ten trees use random tests that never look at the data, each tree sees a tenth of the training records, and the budget pays only for noisy class counts in the leaves. That design makes the guarantee easy to prove, but it caps the accuracy at that of a forest of random trees, and no budget lifts the cap. DP-SGD spends its budget on the model's parameters directly, which is why it wins here by a wide margin. So the methods a library implements matter more than its interface.

4.4 DP-SGD in the libraries not run here

TensorFlow Privacy does the same job as Opacus for Keras, TensorFlow's model-building interface, with an older design. It replaces the optimizer with one that clips and adds noise. It needs the loss, the error that training reduces, per record rather than averaged over the batch, which the `reduction` argument arranges, and the accounting is a separate function call. The code below

follows its documentation for version 0.9.0 and was not run, because that version does not install with the TensorFlow and Python versions used here:

```
from tensorflow_privacy.privacy.analysis.compute_dp_sgd_privacy_lib import
 ↪  compute_dp_sgd_privacy_statement
from tensorflow_privacy.privacy.optimizers.dp_optimizer_keras import
 ↪  DPKerasSGDOptimizer

optimizer = DPKerasSGDOptimizer(l2_norm_clip=1.0, noise_multiplier=3.145,
 ↪  num_microbatches=32, learning_rate=0.5)
model.compile(optimizer=optimizer,
 ↪  loss=tf.keras.losses.BinaryCrossentropy(reduction=tf.losses.Reduction.NONE))
model.fit(x_train, y_train, epochs=20, batch_size=32)
print(compute_dp_sgd_privacy_statement(number_of_examples=455, batch_size=32,
 ↪  num_epochs=20,
                                       noise_multiplier=3.145, delta=1e-3))
```

Two things differ from Opacus in substance. The optimizer clips in microbatches, groups of records whose gradients are averaged before clipping. When the number of microbatches equals the batch size, as above, each group is one record and the guarantee holds. A smaller number saves memory but no longer bounds one record's influence, and the library leaves that choice to you. And the privacy statement assumes Poisson sampling, while `model.fit` shuffles the records and takes fixed batches, so the number it prints describes a different algorithm from the one that ran.

JAX Privacy is the most recent design. It separates batch selection, clipping, noise and accounting into components that you compose, which makes room for newer methods such as noise that is correlated across steps. It also ships an auditing module, which plants made-up records, called canaries, in the training data and measures how well they can be recovered from the model.

5 Job 3: account one training run

The third job asks how far apart the accountants are on one training run, and it needs no data. The run has a noise multiplier of 1, a sampling rate of one in fifteen and 300 steps, and every accountant of the two libraries gives its ε at $\delta = 10^{-3}$. The run has one true ε, the smallest value for which Definition 1.1 holds, but it has no closed form. Each accountant either bounds it from above by a theorem or brackets it by a numerical method, so the spread between them is the spread of the methods.

```
def accountants(sigma: float, q: float, steps: int, delta: float, grid: float =
 ↪  PLD_GRID) -> dict[str, float]:
    """Epsilon for one DP-SGD run (noise multiplier sigma, sampling rate q, `steps`
     ↪  steps) by every accountant the
    two libraries offer, plus the PLD's lower estimate, which no valid upper bound may
 ↪  fall below."""
    out: dict[str, float] = {}
    for name, accountant in [("Opacus RDP", RDPAccountant()), ("Opacus PRV",
     ↪  PRVAccountant())]:
        accountant.history = [(sigma, q, steps)]  # what `steps` calls of .step() would
 ↪  have recorded
```

```
        out[name] = accountant.get_epsilon(delta)
    gdp = GaussianAccountant()
    gdp.history = [(sigma, q, steps)]
    out["Opacus GDP"] = gdp.get_epsilon(delta)
    run = SelfComposedDpEvent(PoissonSampledDpEvent(q, GaussianDpEvent(sigma)), steps)
    out["dp-accounting RDP"] = RdpAccountant().compose(run).get_epsilon(delta)
    out["dp-accounting PLD"] =
↪   PLDAccountant(value_discretization_interval=grid).compose(run).get_epsilon(delta)
    optimistic = privacy_loss_distribution.from_gaussian_mechanism(
        sigma, sampling_prob=q, pessimistic_estimate=False,
↪   value_discretization_interval=grid, use_connect_dots=False
    )
    out["PLD lower estimate"] =
↪   optimistic.self_compose(steps).get_epsilon_for_delta(delta)
    return out
```

The two libraries describe a run differently. An Opacus accountant keeps a history of steps, each a triple of noise multiplier, sampling rate and step count. The library dp-accounting instead builds the run as an event from the inside out: a Gaussian mechanism, applied to a Poisson sample at sampling rate q, composed with itself for the number of steps. That vocabulary is richer, which matters when a run is not plain DP-SGD. It can also describe Laplace steps, sampling without replacement, the replace-one neighbouring relation, and the composition of unrelated mechanisms.

Table 8: One DP-SGD run, noise multiplier 1, sampling rate 0.0667, 300 steps, under every accountant the libraries offer, at $\delta = 10^{-3}$.

Accountant	ε	What the number is
Opacus RDP	6.430	upper bound
Opacus PRV	5.571	upper bound, numerical
Opacus GDP	5.283	approximation, not a bound
dp-accounting RDP	6.456	upper bound
dp-accounting PLD	5.560	upper bound, numerical
PLD lower estimate	5.545	lower bound

The accountants fall into three groups. All of them work with the privacy loss defined before Equation 7, the logarithm of the ratio of the two output densities, summed over the steps. The Rényi differential privacy (RDP) accountants, one per library, summarize that random variable by a family of moments indexed by an order α. They add the moments across steps and convert the best order into an ε. Both give about 6.4, with a small difference because they try different sets of orders.

The second group is the privacy loss distribution (PLD) accountant of dp-accounting and the privacy loss random variable (PRV) accountant of Opacus. Each computes the distribution of the total privacy loss numerically, on a grid of values, and reads off the ε at which the mass beyond it is δ. Both give 5.56 to 5.57, and these are the tight ones. The rounding of each value onto the grid can go up or down. Rounding up gives a certified upper bound, which is what the accountant

reports, and rounding down gives a certified lower bound, which is the PLD's lower estimate of 5.545. So the true ε lies between 5.545 and 5.560, and the RDP bound is 16% above it.

The third group has one member. Opacus's Gaussian differential privacy (GDP) accountant gives 5.28, which is below the certified lower bound, so it cannot be an upper bound on the run's ε. It approximates the total privacy loss by a normal distribution, by a central limit argument that holds only in a limit the run has not reached, and both Opacus's documentation and its warning say so. A paper that reported this value would claim a guarantee that the run does not have.

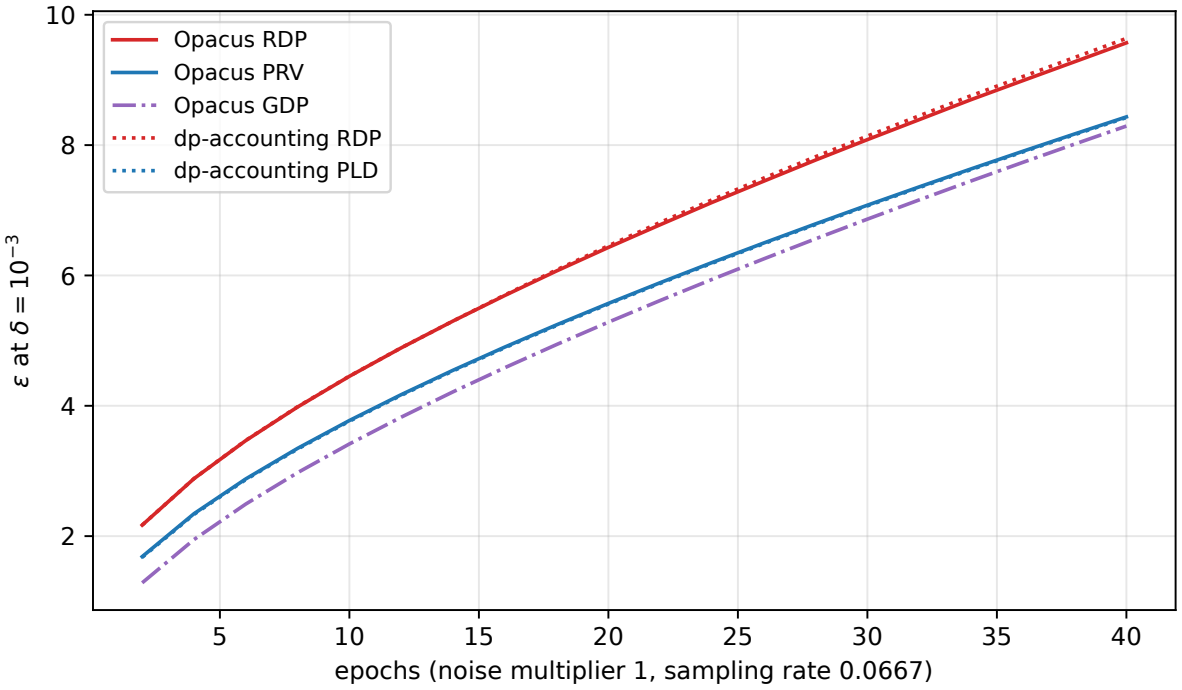


Figure 2: Epsilon against epochs for the same run under five accountants. The two RDP accountants and the two numerical accountants pair up, and the GDP approximation runs below the numerical pair throughout.

Figure 2 extends the comparison to 40 epochs, with the numerical accountants on a coarser grid for speed, to ask whether a longer run changes the picture. It does not. Each pair of curves stays together, the gap between the pairs widens slowly, and the GDP approximation stays below the numerical curves throughout.

So calibrate and report with a numerical accountant, and cross-check it with the other library's. Never report the GDP accountant's value. Opacus's privacy engine defaults to the PRV accountant, but its `get_noise_multiplier` defaults to RDP. So pass `accountant="prv"` to the calibration, as the script does, or you add more noise than the reported guarantee needs.

6 The rest of the toolbox

The packages in Table 5 were not run for this note. For each one, this section says what it is for and what its documentation discloses, which is enough to decide whether to open it.

6.1 Synthetic data

A synthetic dataset generated under differential privacy can be shared and analysed without further accounting, because any analysis of it is post-processing. SmartNoise Synth, from the OpenDP project, wraps the generators that have won the public benchmarks for tabular data. MST, AIM and the older multiplicative weights exponential mechanism fit noisy low-order marginals, the counts of each combination of values in a few columns. Its deep generators build on generative adversarial networks (GANs). Continuous columns must be discretized first, and the documentation warns that the discretization must not be learned from the data.

SmartNoise Synth's release of April 2026 pins Opacus below version 0.15, which is years old, so it cannot be installed next to the Opacus used here. OpenDP's own 0.16.0 release added AIM through an optional extra, which may become the maintained path. synthcity is a research toolkit with dozens of generators, a few of which take a privacy parameter, and its old pins of PyTorch and NumPy mean it needs its own environment too.

6.2 Auditing

An auditing package attacks a trained model to measure, rather than bound, what it leaks. The standard attack is membership inference: given a record, decide whether it was in the training set, usually from the model's loss on it. `tensorflow-empirical-privacy` runs a set of simple attacks on losses or logits, and reports their area under the curve and an empirical lower bound on ε. ML Privacy Meter, from the National University of Singapore, is the research tool for the stronger attacks that train shadow models, and its current form also audits DP lower bounds. But its package on the index is from 2023, and its repository tells you to clone it.

The Adversarial Robustness Toolbox covers membership, attribute and model-inversion attacks across every major framework, as one part of a much larger security library. Google's DP-Auditorium tests a mechanism rather than a model, with a search for neighbouring datasets that violate the claimed guarantee. It lives in the same repository as dp-accounting and is not on the package index.

6.3 Encrypted computation

Encryption protects a computation from the party that runs it, which differential privacy does not do. But it protects nothing about what the result reveals, which differential privacy does, so the two are complementary. TenSEAL exposes Microsoft SEAL's homomorphic encryption on tensors, with the integer scheme BFV and the approximate scheme CKKS. You set its parameters with their SEAL meaning, and they fix the security level and the depth of computation. CrypTen made secure multi-party computation look like PyTorch, but its repository is archived, and its README calls it a research framework that is not production ready.

Concrete ML compiles quantized scikit-learn and PyTorch models to fully homomorphic circuits. Its licence requires a commercial patent licence from Zama for commercial use, which rules it out of many deployments before any technical question arises.

PySyft was once a federated-learning library. Its version 0.10 is a system to submit computations that a data owner runs on data the scientist never sees. MP-SPDZ is the research framework that

runs one program under many multi-party protocols. It comes as source and binaries rather than as a package, and its README says it has not had the security review that production code needs.

6.4 Federated learning

Federated learning keeps data on the devices or sites that hold it and trains on model updates instead. On its own it is not a privacy guarantee, because gradient-inversion attacks recover training data from updates. So what counts in a federated library is the protection it adds. Flower is the most active of the three. It ships central and local differential privacy, an RDP accountant, and secure aggregation, which hides individual updates from the server. TensorFlow Federated has had no release since September 2024, although its repository still receives automated commits. NVIDIA FLARE is an enterprise framework with filters for differential privacy, homomorphic encryption and private set intersection.

6.5 Anonymization

The older way to release a dataset is to generalize and suppress its values until each person hides in a group. The package anjana applies k-anonymity and its relatives, such as ℓ-diversity and t-closeness, to a pandas dataframe. You give it the quasi-identifiers and a hierarchy of generalizations for each. pyCANON measures which of those properties a dataset already has. Neither offers the guarantee of Definition 1.1, which holds against any attacker with any side information, and that is why the rest of this note is about differential privacy. But some regulations still name this syntactic anonymity, and these are the maintained Python tools for it.

7 How to choose

Table 9 gives the choice by job, for someone who starts today. The second column is the default, and the third says when to look elsewhere.

Table 9: Which package for which job.

Job	Start with	Look elsewhere when
Statistics and tables on one machine	OpenDP, through its `Context` and Polars interfaces	You need Spark, in which case Tumult Analytics; or Beam, in which case PipelineDP
Quick experiments and teaching with NumPy arrays	diffprivlib	Anything that ships, because its release says research and education, and because of Section 4's failure
Training a PyTorch model	Opacus	Your stack is JAX, in which case JAX Privacy from its repository; TensorFlow Privacy only if you can stay on TensorFlow 2.15

Job	Start with	Look elsewhere when
Reporting or reviewing an ε	dp-accounting's PLD accountant, cross-checked against Opacus's PRV	The mechanism is not Gaussian noise on Poisson samples, in which case describe it as a `DpEvent` and check that the accountant supports it
Private synthetic tabular data	SmartNoise Synth in its own environment, or OpenDP's AIM extra	Your columns are continuous and you cannot discretize them from public knowledge
Measuring what a finished model leaks	ML Privacy Meter from its repository, or the Adversarial Robustness Toolbox	You need a bound rather than a measurement, in which case an accountant
Computation that the server must not see	TenSEAL for encrypted inference on tensors	You need multi-party protocols, in which case MP-SPDZ, with its own security review
Training across sites	Flower with its central DP strategy and secure aggregation	You already run NVIDIA FLARE
A k-anonymous release that a regulation names	anjana, measured with pyCANON	The release must hold against side information, in which case differential privacy

Whatever you pick, check the four duties of Section 1 in it before the first release, and check its age. The three jobs show where to look.

1. **Find where the unit of privacy and the bounds are declared.** OpenDP, Tumult and PipelineDP make the unit of privacy an argument. diffprivlib and Opacus assume one record per person, so if a person contributes several records, you must bound that yourself. Every bound must be public, and a mean needs a public n or a noisy count. Section 3 showed that if you declare n public with the wrong neighbouring relation, the noise disappears.
2. **Read the noise mechanism's documentation**, not only its scale. Integer noise, truncation and floating-point safety all change what a release looks like, and Section 3 found two of the three in a two-query job.
3. **Name the accountant and its version** in anything you report. Calibrate with the accountant you will report, and cross-check it with the other library's. Section 5 found a 16% spread between valid bounds, and one accountant that is not a bound.
4. **Make sure the library refuses.** In diffprivlib, pass the same `BudgetAccountant` to every call, because a call without it spends from a budget with no limit.
5. **Check the release date against your stack.** A library that wraps PyTorch, TensorFlow or scikit-learn ages with them. Of the packages in this note, three on the package index cannot be installed with a current Python, and one that installs cannot construct its main model.

8 Sources

- Wolberg, W., Street, N. and Mangasarian, O. (1995). Breast Cancer Wisconsin (Diagnostic). UCI Machine Learning Repository. Licence CC BY 4.0. Loaded through scikit-learn, whose description supplies the public bounds.
- Dwork, C., McSherry, F., Nissim, K. and Smith, A. (2006). Calibrating Noise to Sensitivity in Private Data Analysis. *TCC 2006*. Differential privacy, sensitivity and the Laplace mechanism.
- Balle, B. and Wang, Y.-X. (2018). Improving the Gaussian Mechanism for Differential Privacy: Analytical Calibration and Optimal Denoising. *ICML 2018*. The exact condition Equation 7.
- Mironov, I. (2012). On Significance of the Least Significant Bits for Differential Privacy. *ACM CCS 2012*. The floating-point attack on the Laplace mechanism.
- Abadi, M. et al. (2016). Deep Learning with Differential Privacy. *ACM CCS 2016*. DP-SGD and the moments accountant.
- Gaboardi, M., Hay, M. and Vadhan, S. (2020). A Programming Framework for OpenDP. The transformations-and-measurements framework behind OpenDP and Tumult Core.
- Holohan, N., Braghin, S., Mac Aonghusa, P. and Levacher, K. (2019). Diffprivlib: The IBM Differential Privacy Library.
- Yousefpour, A. et al. (2021). Opacus: User-Friendly Differential Privacy Library in PyTorch.
- Gopi, S., Lee, Y. T. and Wutschitz, L. (2021). Numerical Composition of Differential Privacy. *NeurIPS 2021*. The PRV accountant.
- Koskela, A., Jälkö, J. and Honkela, A. (2020). Computing Tight Differential Privacy Guarantees Using FFT. *AISTATS 2020*. The PLD accountant.
- Mironov, I. (2017). Rényi Differential Privacy. *IEEE CSF 2017*. The RDP accountant.
- Ponomareva, N. et al. (2023). How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy. *Journal of Artificial Intelligence Research* 77. The δ rule and what to report.
- Near, J., Darais, D., Lefkovitz, N. and Howarth, G. (2025). Guidelines for Evaluating Differential Privacy Guarantees. NIST SP 800-226. What a reviewer asks of a claim.
- Balle, B. et al. (2026). JAX Privacy: Algorithms for Privacy-Preserving Machine Learning in JAX. The library's design.
- The package pages on the Python Package Index and the repositories of each package, read on 23 October 2026: opendp, diffprivlib, opacus, differential-privacy (dp-accounting and DP-Auditorium), PipelineDP, tumult-analytics, PyDP, smartnoise-sdk, tensorflow/privacy, jax_privacy, synthcity, ml_privacy_meter, adversarial-robustness-toolbox, TenSEAL, CrypTen, concrete-ml, PySyft, MP-SPDZ, flower, tensorflow-federated, NVFlare, anjana and pycanon.